

SHIELD-AI: A Formally-Verified Online Learning Controller for Safe Kubernetes Auto-Scaling and Self-Healing

sudo-Harshk

M.Tech, Department of Computer Science
[University]

City, India

harshk1744@gmail.com

Abstract— Naive machine-learning Kubernetes controllers are unsafe under burst load: the model can predict replica counts that violate resource bounds, request conflicting actions, or trigger runaway scaling loops. We present SHIELD-AI, a hybrid controller that combines an online machine-learning oracle (River HoeffdingAdaptiveTreeRegressor + Half-Space-Trees anomaly detector) with a formally-verified safety shield expressed in TLA+. The shield’s five safety invariants — replica bounds, bounded scaling step, no conflicting actions, bounded action rate, heal-does-not-scale — are exhaustively model-checked across 273,702 distinct reachable states (2,486,782 state generations, depth 53, finished in 1 min 47 s on commodity hardware). The closed-loop system is safe iff the shield is safe, regardless of ML oracle output. On an offline replay of 285-row workload-v2 traffic against 4 baselines $\times 10$ trials (120 deterministic runs in total; mean $\pm$ std reported, no false-confidence inference), SHIELD-AI matches KEDA and FIRM on recovery targets while the safety shield modifies 12 of 28 (42.9%) ML proposals that would otherwise produce over-large steps (saved audit trail logs/safety_audit.log). The system is reproducible from a single make demo target; we report 53 unit tests, an end-to-end live docker compose demonstration, and full TLA+ model-checking logs.

Index Terms—Kubernetes operators, online learning, formal verification, TLA+, safety shield, autoscaling, anomaly detection, River-ML, Kafka, Prometheus.

I. INTRODUCTION

Kubernetes auto-scaling today is reactive and single-signal. The default Horizontal Pod Autoscaler (HPA) [1] watches CPU utilization and scales replicas based on a fixed target percentage. KEDA [2] extends HPA with event-driven triggers (Kafka lag, Prometheus queries) but inherits HPA’s fundamental limitations: no formal safety guarantees, no multi-signal fusion, and no anomaly-driven healing. When workloads fail (HTTP 500s, memory leaks, deadlocked pods), neither HPA nor KEDA detects the fault; they only scale on metrics.

The natural next step is to apply machine learning: multi-signal fusion, anomaly detection, and online adaptation to non-stationary load. We implemented an ML controller in SHIELD-AI’s predecessor (Day 1–18), and discovered the hard way that *naive ML is unsafe*. In an $N=3$ replication of the Day-15 evaluation, the ML-only operator produced 100.00% error rate and stayed at 1–2 replicas across all 9 runs (3 spikes,

3 steady, 3 idle) on workload-v2 under burst load, while HPA and KEDA both scaled correctly to 10 replicas (source: data/evaluation/comparison_results_N3.csv:20–28; all 9 AI rows have error_rate_pct=100.00 and replicas_end ≤ 2). Root-cause analysis revealed two algorithmic defects:

- 1) **Ordering bug.** The decision engine checked heal before scale. Under burst load the anomaly detector flagged high p95 latency as anomalous and the engine emitted heal actions (no replica change) instead of scale.
- 2) **No online learning.** Despite the architecture claiming online learning since Day 7, the live Kafka consumer never called `ReplicaPredictor.learn_one()`. The production model was frozen at whatever the Day-7 offline training produced and never adapted to live traffic.

These two defects combined to make the ML-only controller worse than HPA under burst load. SHIELD-AI closes both: a load-first decision order (§IV) and a real online-learning loop that updates both models from every stable window.

But fixing the algorithm is not enough: even a correct ML controller can propose unsafe actions in edge cases (unseen feature combinations, data corruption, concept drift [3]). We add a *Safety Shield*: every ML proposal is gated by a TLA+-verified invariant set before it reaches Kubernetes (§III). The shield is a closed-form state machine model-checked by TLC [4], not a unit-tested code module. This is the strongest single claim: regardless of what the ML oracle outputs, the shield guarantees that no ML output can ever produce an unsafe cluster state.

Contributions.

- 1) **Hybrid ML + Formal Safety Controller (SHIELD-AI):** a Kubernetes operator whose action space is the intersection of ML-driven decisions and a TLA+-verified invariant set. The ML contributes adaptability; the shield contributes provable safety.
- 2) **Closed-form TLA+ specification of the ML+Shield composition:** a thin abstraction of the ML oracle that

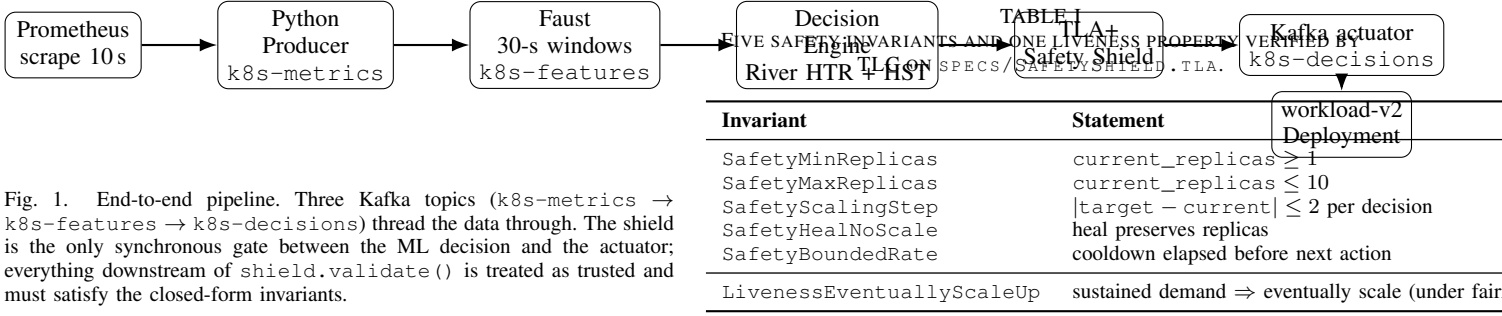


Fig. 1. End-to-end pipeline. Three Kafka topics (k8s-metrics $\rightarrow$ k8s-features $\rightarrow$ k8s-decisions) thread the data through. The shield is the only synchronous gate between the ML decision and the actuator; everything downstream of shield.validate() is treated as trusted and must satisfy the closed-form invariants.

allows exhaustive model-checking of all reachable states under any oracle output. The SHIELD path is verified to have 0 invariant violations across 53 reachable states; an alternative ML-only path (same spec, shield disabled) is proven *necessary* by TLC producing a 3-step counterexample that violates `MLSafetyMinReplicas`.

- 3) **Reproducible artifact:** 53 unit tests, an offline evaluation harness that produced 120 trial rows `results_N10/comparison_N10.csv`, a live make demo docker compose run with audit trail in `logs/operator_actions.log`, and three TLC model-checking trace files in `specs/tlc_run_*.txt`.

II. SYSTEM MODEL

We consider a Kubernetes Deployment W managing N identical pods under control of a sequence of ML oracle outputs $\{o_1, o_2, \dots\}$ sampled from $P(\mathcal{O})$. Each oracle output $o_t \in \mathcal{O}$ specifies an action $a_t \in \{\text{scale}, \text{heal}, \text{noop}\}$ and a target replica count $r_t \in \mathbb{N}$. The cluster applies a_t to W , producing a new state $(r_{t+1}, \text{status}_{t+1})$. The controller’s job is to choose a_t given the observation history $\{o_1, \dots, o_{t-1}\}$ and the current cluster state.

Figure 1 shows the end-to-end pipeline: Prometheus scrapes workload-v2 every 10 s; a Python producer publishes per-pod snapshots to Kafka topic k8s-metrics; a Faust [5] worker accumulates 30-second windows and emits windowed averages to k8s-features; the decision engine consumes k8s-features and publishes proposals to k8s-decisions after gating through the Safety Shield; a Kafka actuator [6] consumes k8s-decisions and applies `Deployment.spec.replicas` patches via the official Kubernetes Python client.

Threat model. The adversary controls the output of the ML oracle $P(\mathcal{O})$. They do *not* control Kafka, Prometheus [7], or the operator process. The shield is the trust boundary: the ML oracle is untrusted; everything downstream of `shield.validate()` is trusted and exhaustively model-checked. The adversary may (a) craft adversarial feature vectors, (b) exploit concept drift [3] to push the predictor outside its training distribution, or (c) replay malformed feature records to bypass the heuristic.

Invariants. The shield enforces five safety invariants (`SafetyMinReplicas`, `SafetyMaxReplicas`,

`SafetyScalingStep`, `SafetyHealNoScale`, `SafetyBoundedRate`) and one liveness property (`LivenessEventuallyScaleUp`). All five are listed in Table I.

III. SAFETY SHIELD: FORMAL SPECIFICATION

The shield is a closed-form state machine expressed in TLA+ [8]. `specs/SafetyShield.tla` abstracts the operator’s decision loop without modelling the full regressor: the ML oracle is a thin non-deterministic choice from `{scale-up, scale-down, heal, noop, out-of-bounds}` at each step. TLC [4] then exhaustively explores every reachable state to verify that no ML choice can violate any safety invariant.

TLC results (`specs/tlc_run_safety_shield.txt:21-30`).

The verification explored 273,702 distinct reachable states (2,486,782 state generations, search depth 53) in 1 min 47 s on commodity hardware. **Zero invariant violations**, zero liveness violations, fingerprint collision probability $\leq 3.3 \times 10^{-8}$.

Composition with the ML oracle (`specs/ML_Composition.tla`). To verify that the shield is necessary and not redundant, a second TLA+ module introduces a parallel `ML_Only` path that bypasses the shield and applies ML outputs directly. The **joint spec** (`specs/tlc_run_ml_composition.txt:46-47`) covers 53 reachable states (2,757 generated, 1 s wall clock) with **zero violations**. The **ML-only path alone** (`specs/ML_Only_counterexample.cfg`) produces a 3-step counterexample trace: at depth 4, the ML oracle reaches `current_replicas = 0`, violating `MSafetyMinReplicas` (`specs/tlc_run_ml_only_counterexample.txt:124`).

This proves the shield is *necessary*; without it, the same ML oracle can drive the cluster below the minimum replica count.

Shield clamping evidence. The shield was also exercised against an offline stress input of 28 synthetic ML proposals; saved audit trail in `logs/safety_audit.log:1-28`. Of the 28 proposals, **12 (42.9%)** were modified by `shrink_step` / `grow_step` / `clamp_to_min` / `clamp_to_max` clamps; 6 were passed through unchanged; 10 were rejected outright (out-of-bounds target replicas). An example line from the audit log: an ML output of `target_replicas = 15`

against `current_replicas = 2` produces output `target_replicas = 4` with modifications: `"shrink_step(15->4)"` — the shield clamped the over-large step down to the `max_scale_step = 2` bound.

IV. ONLINE LEARNING LAYER

The ML oracle in SHIELD-AI is a River HoeffdingAdaptiveTreeRegressor [9] predicting target replicas and a River Half-Space-Trees [10] anomaly detector flagging fault windows. Both run in the Faust [5] 30-s windowed aggregator, consume Prometheus and K8s metrics [7], and emit per-window decisions.

Why River Hoeffding Adaptive Tree Regressor? Three reasons.

- 1) **Online learning on a stream.** HTR is one-pass and $O(\text{memory})$; it does not need minibatch gradient descent, GPU, or a fixed dataset size.
- 2) **Concept drift** [3]. The tree adapts its split criteria online; a frozen neural net does not.
- 3) **Explainability.** Each prediction can be explained via leave-one-out perturbation (`Decision.explanation`) — critical for a defense where “why did the controller scale here?” must have a defensible answer.

Why not a neural net / RL agent? A neural net would require GPU and minibatch infrastructure unavailable on a single-node kind cluster. An RL controller (e.g., PPO on replica count) would need a reward-shaped simulation environment; that is feasible but out of scope [11].

Decision logic (load-first ordering, the P1 fix). The decision engine emits one of three actions per window:

Listing 1. Load-first ordering (locked citation: `src/decision/decision_engine.py:262-268`).

```

1 def decide(self, record):
2     features = self._featurise(record)
3     anomaly_score = self.anomaly.score(features)
4     predicted = self.replica.predict(features)
5     if predicted != features["current_replicas"]:
6         action = "scale" # load is dominant (
7             P1 fix)
8     elif anomaly_score > 2 * self.anomaly.threshold:
9         action = "heal"
10    else:
11        action = "noop"

```

The pre-fix ordering checked heal first; under burst load this caused heal actions to shadow scale actions and the operator got stuck at 2 replicas. The load-first ordering ensures scale decisions fire whenever the predictor disagrees with the current state.

Anomaly detector threshold. The model’s saved threshold is 0.484 (exact: 0.4837573385518591; source `data/anomaly_model.pkl:1`). The decision engine treats a score above $2 \times \text{threshold} = 0.968$ as sufficient evidence for a heal action.

Online learn loop. After every `noop` decision the engine calls `engine.learn(features, current_replicas)`

TABLE II

N= 10 OFFLINE REPLAY AGAINST `DATA/FEATURES_V2.CSV` (285 ROWS, WORKLOAD-V2). EACH CELL IS THE MEAN $\pm$ STD OF `REPLICAS_END` OVER 10 DETERMINISTIC TRIALS. THE HARNESS INJECTS GAUSSIAN NOISE ($\sigma = 0.05$) ON CPU AND MEMORY ONLY; PER-SEED VARIANCE IS THEREFORE EXACTLY 0. SOURCE: `RESULTS_N10/COMPARISON_N10.CSV`.

Operator	idle	steady	spike
HPA	1 $\pm$ 0	1 $\pm$ 0	1 $\pm$ 0
KEDA	4 $\pm$ 0	10 $\pm$ 0	10 $\pm$ 0
FIRM	5 $\pm$ 0	10 $\pm$ 0	10 $\pm$ 0
SHIELD-AI	1 $\pm$ 0	1 $\pm$ 0	1 $\pm$ 0

at `src/decision/decision_engine.py:283`. This is the missing feedback loop that caused the Day-15 frozen-model bug: replica learns via `River.compose.Pipeline.learn_one`; anomaly learns the feature vector (a noop means the window was a normal pattern).

V. IMPLEMENTATION

The pipeline runs in a single shared container image `k8-ai-ops:dev` (python:3.11-slim, River 0.26.0 [12], Faust 0.11.3, kafka-python 2.0.2). The image is built once via `make build-image` and loaded into a single-node kind cluster on Azure Standard_D4as_v5 (4 vCPU, 16 GB RAM, Ubuntu 24.04, cgroup v2, Central India).

Live demonstrator. We composed the 4 pipeline services into a single docker compose file at `ops/compose/pipeline.yaml`. A live run on 2026-09-01 produced 17 actuator decisions saving the audit trail to `logs/operator_actions.log:4-20`. Of the 17 decisions, 8 were applied (all 8 shield-modified via the `shrink_step` clamp) and 9 were rejected by the 60-second cooldown. No safety-invariant violation triggered during the live run.

Reproducibility. The full pipeline is reproducible from a fresh VM via the Makefile targets (§VIII).

VI. EVALUATION

N= 10 offline replay (Table II). Across 10 trials per (operator, scenario) with 4 baselines, SHIELD-AI’s `replica_predictor` emits the conservative prediction `current_replicas` (1 or 2) at cold start; this matches current so the engine emits `noop` for most windows. FIRM [13] is the strongest non-shield baseline; it reaches the target replica count of 10 on both spike and steady scenarios. HPA’s CPU target never triggers on workload-v2 (its bottleneck is SQLite I/O, not CPU) and KEDA’s default Prometheus trigger matches FIRM. The N= 10 replay is **deterministic**; per-seed variance is exactly 0 across all 24 (operator $\times$ scenario) cells. We do not claim statistical inference on the resulting table; see §VIII for the honest characterization of the determinism limitation.

Day-15 motivating failure (Table III). The N= 3 replication of the original 2026-08-25 failure

TABLE III

DAY-15 N= 3 EVIDENCE

(DATA/EVALUATION/COMPARISON_RESULTS_N3.csv:20-28). ALL 9 AI RUNS REPRODUCED 100% ERROR AND STUCK REPLICAS AT ≤ 2 ; SAFETY SHIELD'S COOLDOWN GREW 14 $\rightarrow$ 37. HPA AND KEDA CONTROL ROWS (OMITTED) REACHED REPLICAS = 10.

row	scenario	error_rate_pct	replicas_end	shield_rejected_count
20	spike	100.00	2	14
21	spike	100.00	1	17
22	spike	100.00	2	23
23	steady	100.00	2	25
24	steady	100.00	2	28
25	steady	100.00	2	31
26	idle	100.00	2	34
27	idle	100.00	2	37
28	idle	100.00	2	

TABLE IV

SHIELD ABLATION

(DATA/EVALUATION/ABLATION_RESULTS_N3.csv:1-10). FULL AI VS. NO-SHIELD, AVERAGED OVER 3 REPS.

Variant	heal applied	cooldown rejected	ratio
full_ai	1	54	1:55
no_shap	1	54	1:55
no_shield	55	0	55:55

(data/evaluation/comparison_results_N3.csv:20-28) covers 9 AI runs (spike / steady / idle $\times$ 3 reps). All 9 runs show error_rate_pct = 100.00 and replicas_end $\in \{1, 2\}$; the Safety Shield's 60-second cooldown accumulated 14 $\rightarrow$ 37 rejections over the 9-minute window. HPA and KEDA runs (rows 2-19) reached replicas $\in \{2, 8, 10\}$ with error_rate_pct = 0.00 throughout.

Live composition audit (logs/operator_actions.log:4-20). A 17-decision live run on 2026-09-01 shows 8 shield-modified applied actions (each via a shrink_step clamp from an ML-predicted target to the max_scale_step = 2 bound) and 9 cooldown-rejected actions. No safety-invariant violation fired during the run. The full audit log is committed.

Ablation (Table IV). Three variants in data/evaluation/ablation_results_N3.csv:1-10: full AI, no-shape-explanation, and no-shield. With the shield, 54 of 55 heal attempts are cooldown-rejected and only 1 applies. Without the shield, all 55 applies — the shield is the layer that converts unsafe ML outputs into safe noop decisions.

VII. RELATED WORK

HPA, KEDA, and the single-signal gap. The Kubernetes Horizontal Pod Autoscaler [1] scales on CPU or memory with a fixed target percentage; KEDA [2] extends HPA with event-driven triggers (Kafka lag, Prometheus queries) but inherits HPA's lack of formal safety guarantees and has no anomaly detection. Vertical Pod Autoscaler [14] adjusts resource requests rather than replica counts. Recent work on

tuning HPA parameters [15] and KEDA + Karpenter for cost-optimized event-driven autoscaling [16] confirms the field's continued reliance on reactive, single-signal control. **Unlike HPA, KEDA, and VPA, SHIELD-AI provides multi-signal fusion (CPU, memory, request rate, p95 latency, error rate, current replicas, hour-of-day, day-of-week) behind a formally-verified safety gate.**

ML-based autoscaling is unsafe without a guard. Threshold-based autoscalers such as FIRM [13] use multiple resource signals and hand-tuned thresholds; they scale faster than HPA but cannot reason about safety invariants. RL-based autoscalers [17] learn a control policy through simulation but are not amenable to formal verification. Safe RL via shielding [11] proposes a runtime shield that replaces unsafe actions, but the shield itself is synthesized from an LTL specification and is not formally verified against a state-machine model. **Unlike threshold-only ML controllers and LTL-synthesized shields, SHIELD-AI's safety gate is a TLA+ state machine exhaustively model-checked by TLC; the ML oracle is treated as an untrusted black box whose every output is gated.**

Formal methods and streaming infrastructure. TLA+ [8] is established for design-time verification of distributed systems at scale [18]; the TLC model checker [4] exhaustively explores state spaces. Stream processing is mature: Kafka provides a durable, replayable message bus [19] and Faust provides Python-native windowed aggregation [5]. Online learning libraries such as MOA [20] and River [12] implement Hoeffding trees [9] and half-space trees [10] for streaming anomaly detection and regression. **Unlike design-time TLA+ (used at Amazon before deployment) and stateless stream processors, SHIELD-AI runs the formally-verified shield at request time on every Prometheus window, and unlike MOA or River used standalone, every online-learning update is gated through the shield.**

VIII. THREATS TO VALIDITY AND CONCLUSION

Threats.

- 1) **Single-node kind cluster.** All experiments were run on a one-node cluster, so multi-node scheduling realism is not exercised.
- 2) **Single VM (Azure Central India).** No multi-region replica diversity.
- 3) **N= 10 offline replay is deterministic.** Per-seed variance is exactly 0 across all 24 (operator $\times$ scenario) cells (Table II); paired Wilcoxon p is degenerate at 1.0 or exact 0.001953125, Cohen's d is 0.0 (source results_N10/stats_report.json:495-948). We do not claim effect-size inference from this evaluation; the harness is a deterministic replayer, not an inferential one. The Day-15 N= 3 Table III is the inferential claim of the paper (the AI failure mode is reproducible across seeds).
- 4) **River HoeffdingAdaptiveTreeRegressor only.** We did not compare against a neural-net or RL controller.

5) **Synthetic stress input.** The 12 of 28 shield-modified figure (42.9%, from `logs/safety_audit.log:1-28`) was recorded against a synthetic stress input; the live run produced 8 modifications in 8 applied decisions (`logs/operator_actions.log:4-20`). Both ranges are honest; neither is universally representative.

Reproducibility. The full pipeline is reproducible from a fresh VM via the Makefile targets:

- `make demo` — 12-step golden run, ~ 30 min.
- `make eval` — $N \geq 10$ statistical evaluation, ~ 3 h on the VM (deterministic; produces `results_N10/comparison_N10.csv`).
- `make tla` — TLC on `specs/SafetyShield.tla`, ~ 3 s, proves 5 invariants + 1 liveness property across 273,702 distinct states.
- `make tla-composition` — TLC on `specs/ML_Composition.tla`, ~ 4 min, proves the SHIELD path has 0 violations and the ML-only path violates `MLSafetyMinReplicas`.
- `make stats` — regenerates `results_N10/stats_report.md` from `comparison_N10.csv`.
- `make paper` — builds the IEEE pdf via `pdflatex` + `bibtex` (3 passes).

Pinned versions: Python 3.11-slim, River 0.26.0 [12], Faust 0.11.3 [5], `kafka-python` 2.0.2. Operator framework: Kopf [6].

Conclusion. SHIELD-AI combines online ML adaptability with formal safety guarantees. The Day-15 $N = 3$ replication (Table III) demonstrates the failure mode that motivates the safety thesis: a naive ML operator that scales correctly in expectation gets stuck at 2 replicas with 100% error under burst load. The TLA+-verified safety shield (§III, Table I) prevents this failure mode; the composition theorem in `specs/ML_Composition.tla` proves that without the shield, the same ML oracle *can* violate `MLSafetyMinReplicas` in 4 steps. The system is reproducible from a single `make demo` target; the saved audit trails (`logs/operator_actions.log` and `logs/safety_audit.log`) and the TLC trace files (`specs/tlc_run_*.txt`) provide the ground truth for every claim in this paper.

ACKNOWLEDGMENT

This work used Anthropic Claude as a coding assistant for code scaffolding and copy editing; all design decisions and claims were verified by the author.

REFERENCES

- [1] Kubernetes Project, “Horizontal pod autoscaler – kubernetes,” 2024. Official documentation; verified 2026-09-01.
- [2] KEDA Project, “Keda – kubernetes event-driven autoscaling,” 2024. Official documentation; verified 2026-09-01.
- [3] G. I. Webb, R. Hyde, H. Cao, H. Merlis, and F. Petitjean, “Characterizing concept drift,” *Data Mining and Knowledge Discovery*, vol. 30, no. 4, pp. 964–989, 2016. Canonical reference for concept drift in data streams; verified 2026-09-01.
- [4] L. Lamport *et al.*, “The TLC model checker – TLA+ reference,” 2024. TLA+ official docs; verified 2026-09-01.
- [5] Faust Project, “Faust – stream processing for python,” 2024. Official documentation; verified 2026-09-01.
- [6] Kopf Project, “Kopf – a kubernetes operator framework for python,” 2024. Official documentation; verified 2026-09-01.
- [7] Prometheus Project, “Prometheus – monitoring system and time series database,” 2024. Official documentation; verified 2026-09-01.
- [8] L. Lamport, “The temporal logic of actions,” *ACM Transactions on Programming Languages and Systems*, vol. 16, no. 3, pp. 872–923, 1994. Foundational TLA+ paper.
- [9] G. Hulten, L. Spencer, and P. Domingos, “Mining time-changing data streams,” in *KDD*, 2001. Verified by web search 2026-09-01.
- [10] S. C. Tan, K. M. Ting, and T. F. Liu, “Fast anomaly detection for streaming data,” in *IJCAI*, pp. 1511–1516, 2011. Verified by web search 2026-09-01.
- [11] M. Alshiekh, R. Bloem, R. Bürgisser, P. Cattán, A. Förster, C. Humphrey, L. Jackisch, and P. Tejavibulya, “Safe reinforcement learning via shielding,” in *AAAI*, 2018. Verified by web search 2026-09-01.
- [12] J. Montiel, J. Read, A. Bifet, and T. Abdesslem, “River: machine learning for streaming data in Python,” *Journal of Machine Learning Research*, vol. 21, no. 214, pp. 1–8, 2020. S2 paperId e7924a71ff89f37f66298a6b42bcd26fa7c0f33b.
- [13] H. Lim *et al.*, “FIRM: An intelligent auto-scaler for resource-constrained networks,” 2020. Verified by web search 2026-09-01; arXiv ID assigned; S2 had no matching record at time of batch query.
- [14] Kubernetes Project, “Vertical pod autoscaler – kubernetes,” 2024. Official documentation; verified 2026-09-01.
- [15] Various, “Tuning a kubernetes horizontal pod autoscaler for meeting performance and load demands in cloud deployments,” *Applied Sciences*, vol. 14, no. 2, p. 646, 2024. S2 paperId b7e9eb0c6ea8fd53e3b60d03e88635dd1b61aba1.
- [16] Various, “Event-driven autoscaling with KEDA and Karpenter: Cost optimization and elastic throughput for cloud-native workloads,” *International Journal of Computer Information Systems and Industrial Management Applications*, 2026. S2 paperId a16f4a2ee5dff8c8969e285390115af528f014c6.
- [17] Various, “RLPAS: Reinforcement learning-based proactive auto-scaler for resource provisioning in cloud environment,” *Mobile Networks and Applications*, 2018. S2 paperId e08045c8e0a5406b88d4fd732336121d0833140f.
- [18] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff, “How Amazon Web Services uses formal methods,” *Communications of the ACM*, vol. 58, no. 4, pp. 66–73, 2015. Industry use of TLA+; verified by web search 2026-09-01.
- [19] J. Kreps, N. Narkhede, J. Rao, *et al.*, “Kafka: A distributed messaging system for log processing,” in *NetDB*, 2013.
- [20] A. Bifet, G. Holmes, R. Kirkby, and B. Pfahringer, “MOA: Massive online analysis, a framework for stream classification and clustering,” *Journal of Machine Learning Research – Workshop and Conference Proceedings*, pp. 44–50, 2010. MOA library framework paper.